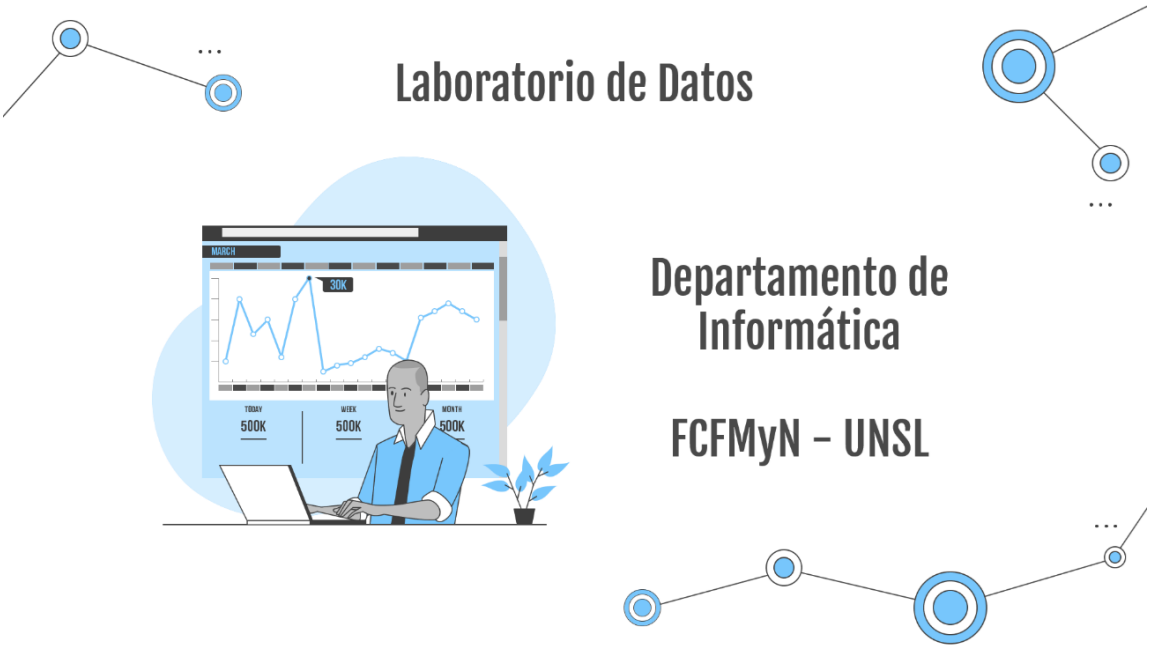
Guía práctica de introducción al Análisis Exploratorio de Datos



Profesores

- Barrionuevo, Mercedes.
- Villegas, Paula.
- Vallejo, Enrique.

Índice

1. Introducción **4**

2. Metodología **5**

3. Análisis Exploratorio de Datos **6**

3.1. Análisis descriptivo 6

3.2. Ajuste de tipos de variables 10

3.3. Detección y tratamiento de datos ausentes 12

3.3.1. Detección de datos ausentes 12

3.3.2. Tratamiento de datos ausentes 15

3.4. Detección y tratamiento de valores atípicos (outliers) 17

3.4.1. Variables continuas 17

3.4.2. Variables categóricas 22

3.5. Análisis de correlación entre variables 25

4. Conclusiones **32**

5. Recursos adicionales **33**

Índice de figuras

1. Etapas de un Análisis Exploratorio de Datos 5
2. Etapa N° 1 - Análisis Descriptivo 5
3. Contenido del dataset 8
4. Estructura interna del dataset 8
5. Resumen general de las variables del dataset 9
6. Histogramas para observar la distribución de cualquier variable numérica. 10
7. Etapa N° 2 - Ajuste de tipos de variables. 10
8. Tipos de datos de las variables del dataset. 12
9. Etapa N° 3 - Detección y tratamiento de **datos ausentes**. 12
10. Detección de valores ausentes. 14
11. Estadísticas de valores ausentes. 15
12. Etapa N° 4 - Identificación de valores atípicos. 17
13. Representación del histograma de la variable O3. 19
14. Representación de un gráfico boxplot para identificar la presencia de valores atípicos. 20
15. Representación de un gráfico boxplot en el que se observa la presencia de valores atípicos y su distribución. Variable O3. 21
16. Estadísticas de la variable O3. 21
17. Antes de eliminar outliers. 23
18. Después de eliminar outliers. 24
19. Distribución por provincia (librería `seaborn`). 24
20. Etapa N° 5 - Correlación de variables. 25
21. Interpretación entre la correlación lineal entre dos variables. 26
22. Matriz de correlación. 29
23. Relaciones entre variables numéricas. 31

El Análisis Exploratorio de Datos (EDA) constituye una fase fundamental en cualquier proyecto de ciencia de datos. Este documento presenta un resumen de la guía práctica original, con el objetivo de proporcionar una introducción accesible pero rigurosa a las técnicas esenciales de EDA utilizando las bibliotecas más comunes del ecosistema Python: `pandas`, `numpy`, `matplotlib`, `seaborn` y `plotly`.

1. Introducción

El Análisis Exploratorio de Datos (EDA) se refiere al conjunto de técnicas estadísticas cuyo objetivo es explorar, describir y resumir la naturaleza de los datos, así como comprender las relaciones existentes entre las variables de interés.

Un EDA está compuesto por la siguinte secuencia secuencia de etapas:

1. Realizar un análisis descriptivo de las variables:
   Nos será útil para lograr la obtención de una idea que nos represente la información contenida en el dataset.
2. Re-ajustar los tipos de las variables:
   Esta etapa nos servirá para asegurar la consistencia al momento de realizar operaciones futuras.
3. Detectar y tratar los datos ausentes (missing data):
   El tratamiento o la eliminación de datos ausentes es esencial, ya que de otra manera no será posible procesar adecuadamente las variables numéricas.
4. Identificar los de datos atípicos (outliers):
   Esta tarea y el posterior tratamiento de los valores atípicos, nos aydará a evitar distorsiones para el caso de posteriores análisis estadísticos.
5. Realizar un examen numérico y graficar las relaciones entre las variables analizadas:
   Esta etapa nos servirá en la determinación del grado de correlación entre las variables, ayudándonos a pronosticar el comportamiento de una variable en función de las otras.

Figura 1: Etapas de un Análisis Exploratorio de Datos

La Figura 1 muestra un esquema de las etapas del análisis exploratorio de datos que se relata en los contenidos de esta guía.

2. Metodología

Para la realización del caso práctico se seleccionó un dataset relacionado con el registro de la calidad del aire en la Comunidad Autónoma de Castilla y León que se encuentra en el **portal de datos abiertos** datos.gob.es. Una vez cargadas las librerías y el conjunto de datos en el entorno de desarrollo procederemos a realizar el EDA siguiendo los pasos que a continuación se proponen.

Figura 2: Etapa N° 1 - Análisis Descriptivo

Se utilizarán las siguientes bibliotecas para trabajar con el lenguaje Python:

```
# Importacion de librerias necesarias
import pandas as pd
import numpy as np
```

```python
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from scipy import stats

# Configuracion de visualizacion
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 20)
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)
```

Listing 1: Importación de librerías necesarias

3. Análisis Exploratorio de Datos

El EDA que se propone seguirá los siguientes pasos:

1. Análisis descriptivo de las variables.
2. Ajuste de los tipos de variables.
3. Detección y tratamiento de datos ausentes.
4. Identificación y tratamiento de valores atípicos.
5. Análisis de correlación entre variables.

3.1. Análisis descriptivo

Una vez descargado el dataset y cargado en el entorno de desarrollo (por ejemplo Google Colab), se deben ejecutar tareas pra la reutilización de los datos, como es el caso de una visualización interactiva o quizás el desarrollo de una aplicación.

El primer contacto con los datos nos debe permitir la comprensión de la estructura, dimensiones y características básicas del dataset.

```python
# Carga del dataset (ejemplo con datos de calidad del aire)
# En un caso real, cargariamos el archivo correspondiente
# df = pd.read_csv('calidad_aire.csv')

# Para este ejemplo, simulamos la estructura del dataset
np.random.seed(42)
fechas = pd.date_range(start='1997-01-01', end='2020-12-31', periods
    =1000)
```

```python

calidad_aire = pd.DataFrame({
    'Fecha': fechas,
    'CO_mg_m3': np.random.normal(0.9, 0.5, 1000),
    'NO_ug_m3': np.random.normal(13.3, 10, 1000),
    'NO2_ug_m3': np.random.normal(21.4, 8, 1000),
    'O3_ug_m3': np.random.normal(53, 15, 1000),
    'PM10_ug_m3': np.random.normal(22.7, 8, 1000),
    'SO2_ug_m3': np.random.normal(9.1, 5, 1000)
})

# Introducimos algunos valores NA para el ejemplo
for col in calidad_aire.columns[1:]:
    mask = np.random.random(1000) < 0.1
    calidad_aire.loc[mask, col] = np.nan

# Funciones descriptivas equivalentes a view(), str() y summary() en R
print('=== PRIMERAS FILAS (head) ===')
print(calidad_aire.head())

print('\n=== INFORMACION DEL DATAFRAME (info) ===')
print(calidad_aire.info())

print('\n=== ESTADISTICAS DESCRIPTIVAS (describe) ===')
print(calidad_aire.describe())

print('\n=== ESTRUCTURA DETALLADA (dtypes) ===')
print(calidad_aire.dtypes)
```

Listing 2: Carga y análisis descriptivo inicial

Con este fin se aplican funciones de estadística descriptiva para explorar la estructura del dataset y examinar los datos y variables que presenta.

```
••• === PRIMERAS FILAS (head) ===
                          Fecha  CO_mg_m3   NO_ug_m3  NO2_ug_m3   O3_ug_m3  \
0 1997-01-01 00:00:00.000000000  1.148357  27.293554  15.998574  24.382887
1 1997-01-09 18:34:14.054054054  0.830868  22.546337  20.243851  40.094225
2 1997-01-18 13:08:28.108108108  1.223844  13.896304  15.060641  46.795917
3 1997-01-27 07:42:42.162162162  1.661515   6.830632  18.936308  81.315315
4 1997-02-05 02:16:56.216216216  0.782923  20.282233   6.251083  61.348297

   PM10_ug_m3  SO2_ug_m3
0         NaN   6.981202
1   22.450372   6.832929
2   22.844135   0.121784
3   26.481043        NaN
4   11.765133  12.764145
```

Figura 3: Contenido del dataset

```
=== INFORMACION DEL DATAFRAME (info) ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Fecha       1000 non-null   datetime64[ns]
 1   CO_mg_m3    895 non-null    float64
 2   NO_ug_m3    904 non-null    float64
 3   NO2_ug_m3   905 non-null    float64
 4   O3_ug_m3    910 non-null    float64
 5   PM10_ug_m3  899 non-null    float64
 6   SO2_ug_m3   897 non-null    float64
dtypes: datetime64[ns](1), float64(6)
memory usage: 54.8 KB
None
```

Figura 4: Estructura interna del dataset

```
=== ESTADISTICAS DESCRIPTIVAS (describe) ===
                      Fecha    CO_mg_m3    NO_ug_m3   NO2_ug_m3    O3_ug_m3  \
count                  1000  895.000000  904.000000  905.000000  910.000000
mean   2008-12-31 12:00:00    0.903825   13.995404   21.454413   52.782758
min    1997-01-01 00:00:00   -0.720634  -16.103886   -2.756097    9.058270
25%    2003-01-01 06:00:00    0.573885    7.243596   16.317101   42.060241
50%    2008-12-31 12:00:00    0.906965   13.883217   21.325597   53.058960
75%    2014-12-31 18:00:00    1.222715   20.498103   26.507135   63.156219
max    2020-12-31 00:00:00    2.826366   45.231076   52.809902  101.646395
std                    NaN    0.489783   10.008315    7.823125   15.379908

       PM10_ug_m3   SO2_ug_m3
count  899.000000  897.000000
mean    22.256488    8.882790
min     -2.713631   -5.397569
25%     17.020463    5.742439
50%     22.521598    8.997197
75%     27.778515   12.161661
max     47.603282   24.591497
std      7.994920    5.001406
```

Figura 5: Resumen general de las variables del dataset

Para la visualización de distribuciones, utilizamos histogramas:

```python
# Histogramas de dos variables
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histograma para O3
axes[0].hist(calidad_aire['O3_ug_m3'].dropna(), bins=50, alpha=0.7,
             color='skyblue', edgecolor='black')
axes[0].set_xlabel('O3 (ug/m3)')
axes[0].set_ylabel('Frecuencia')
axes[0].set_title('Distribucion de O3')
axes[0].set_xlim(0, 150)

# Histograma para PM10
axes[1].hist(calidad_aire['PM10_ug_m3'].dropna(), bins=50, alpha=0.7,
             color='lightcoral', edgecolor='black')
axes[1].set_xlabel('PM10 (ug/m3)')
axes[1].set_ylabel('Frecuencia')
axes[1].set_title('Distribucion de PM10')
axes[1].set_xlim(0, 150)
```

```

plt.tight_layout()
plt.show()
```

Listing 3: Histogramas con matplotlib

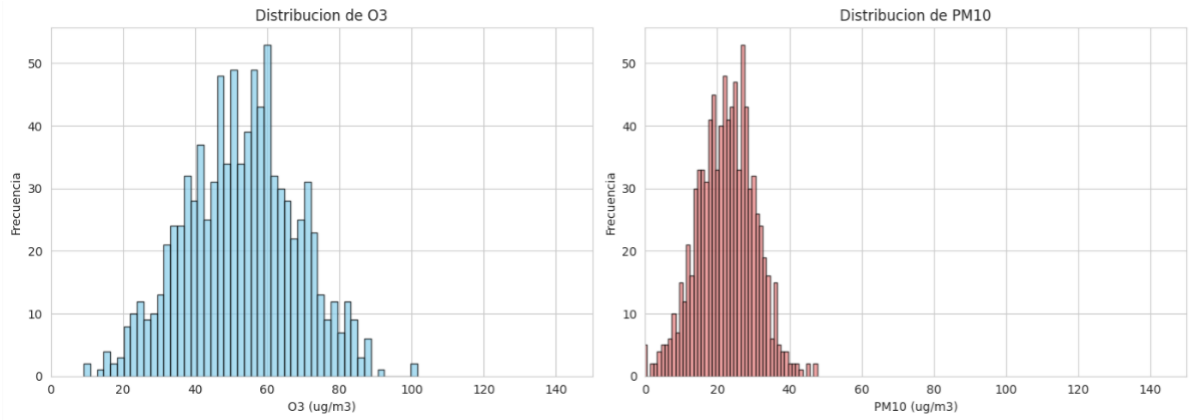


Figura 6: Histogramas para observar la distribución de cualquier variable numérica.

3.2. Ajuste de tipos de variables

Es fundamental verificar que cada variable tenga el tipo de dato adecuado para su correcto procesamiento; para ello puede ser necesario modificar el dataset de manera de realizar una comprobación del efecto que producen sobre los datos los cambios aplicados en cada nueva etapa del EDA.

Figura 7: Etapa N° 2 - Ajuste de tipos de variables.

```
# Verificar tipos actuales
print('Tipos originales:')
print(calidad_aire.dtypes)
```

```python

# Asegurar que Fecha sea tipo datetime
calidad_aire['Fecha'] = pd.to_datetime(calidad_aire['Fecha'])
print('\nTipo de Fecha tras conversion:', calidad_aire['Fecha'].dtypes)

# Para variables categoricas (simulamos Provincia y Estacion)
provincias = ['Burgos', 'Leon', 'Salamanca', 'Zamora', 'Valladolid',
              'Palencia', 'Avila', 'Soria', 'Segovia', 'Madrid']
estaciones = ['Estacion1', 'Estacion2', 'Estacion3', 'Estacion4']

calidad_aire['Provincia'] = np.random.choice(provincias, 1000)
calidad_aire['Estacion'] = np.random.choice(estaciones, 1000)

# Convertir a tipo categorico
calidad_aire['Provincia'] = calidad_aire['Provincia'].astype('category')
calidad_aire['Estacion'] = calidad_aire['Estacion'].astype('category')

print('\nTipos despues de conversion:')
print(calidad_aire[['Fecha', 'Provincia', 'Estacion']].dtypes)

# Ver valores unicos
print('\nProvincias unicas:', calidad_aire['Provincia'].unique())
print('Estaciones unicas:', calidad_aire['Estacion'].unique())
```

Listing 4: Verificación y ajuste de tipos de datos

Los tipos de variables habituales, que puede albergar un dataset pueden ser:

- **numérico:**, almacena números que pueden ser decimales o enteros.
- **carácter**: alberga cadenas de texto.
- **categórico:** contiene un número limitado de valores o categorías de información.
- **lógico o booleano:** variables binarias que solo pueden tomar dos valores: **TRUE** y **FALSE** ó **0** y **1**; pueden ser resultado de una comparación o condición de otras variables presentes en el conjunto de datos.
- **fecha:** almacena intervalos específicos de tiempo.

```
Tipos originales:
Fecha          datetime64[ns]
CO_mg_m3               float64
NO_ug_m3               float64
NO2_ug_m3              float64
O3_ug_m3               float64
PM10_ug_m3             float64
SO2_ug_m3              float64
dtype: object

Tipo de Fecha tras conversion: datetime64[ns]

Tipos despues de conversion:
Fecha          datetime64[ns]
Provincia            category
Estacion             category
dtype: object

Provincias unicas: ['Leon', 'Salamanca', 'Segovia', 'Soria', 'Valladolid', 'Burgos', 'Zamora', 'Palencia', 'Avila', 'Madrid']
Categories (10, object): ['Avila', 'Burgos', 'Leon', 'Madrid', ..., 'Segovia', 'Soria', 'Valladolid',
                          'Zamora']
Estaciones unicas: ['Estacion1', 'Estacion3', 'Estacion4', 'Estacion2']
Categories (4, object): ['Estacion1', 'Estacion2', 'Estacion3', 'Estacion4']
```

Figura 8: Tipos de datos de las variables del dataset.

3.3. Detección y tratamiento de datos ausentes

Figura 9: Etapa N° 3 - Detección y tratamiento de **datos ausentes**.

Una problemática habitual es la presencia de datos ausentes, perdidos, missing values, o celdas vacías, (`NaN` en Python) ya que la mayoría de las veces se debe a fallos en la transcripción de los datos o problemas durante la recopilación de datos, por ejemplo, debido a la imposibilidad para obtener una cierta medida u observación, lo que puede afectar **significativamente** análisis posteriores.

3.3.1. Detección de datos ausentes

```python
# is.na() equivalente en Python
print('=== DETECCION DE VALORES AUSENTES ===')
```

```python

# Detectar valores ausentes (equivalente a is.na())
print('\nPrimeras filas con deteccion de NA:')
print(calidad_aire.head(10).isna())

# any(is.na()) equivalente
print('\nalgun valores ausentes:', calidad_aire.isna().any().any())

# sum(is.na()) equivalente - total de NA
print('Numero total de NA:', calidad_aire.isna().sum().sum())

# mean(is.na()) equivalente - porcentaje de NA
print('Porcentaje de NA:', (calidad_aire.isna().sum().sum() /
                            calidad_aire.size) * 100, '\%')

# Por columnas - numero de NA
print('\nNumero de NA por columna:')
print(calidad_aire.isna().sum())

# Por columnas - porcentaje de NA
print('\nPorcentaje de NA por columna:')
print((calidad_aire.isna().sum() / len(calidad_aire)) * 100)
```

Listing 5: Detección de valores ausentes

```
=== DETECCION DE VALORES AUSENTES ===

Primeras filas con deteccion de NA:
   Fecha  CO_mg_m3  NO_ug_m3  NO2_ug_m3  O3_ug_m3  PM10_ug_m3  SO2_ug_m3  \
0  False     False     False      False     False        True      False
1  False     False     False      False     False       False      False
2  False     False     False      False     False       False      False
3  False     False     False      False     False       False       True
4  False     False     False      False     False       False      False
5  False     False     False      False     False       False      False
6  False     False     False      False     False        True      False
7  False     False     False      False     False       False      False
8  False     False     False      False     False       False       True
9  False     False      True      False     False        True      False

   Provincia  Estacion
0      False     False
1      False     False
2      False     False
3      False     False
4      False     False
5      False     False
6      False     False
7      False     False
8      False     False
9      False     False
```

Figura 10: Detección de valores ausentes.

```
Valores ausentes: True
Numero total de NA: 590
Porcentaje de NA: 6.555555555555556 %

Numero de NA por columna:
Fecha           0
CO_mg_m3      105
NO_ug_m3       96
NO2_ug_m3      95
O3_ug_m3       90
PM10_ug_m3    101
SO2_ug_m3     103
Provincia       0
Estacion        0
dtype: int64

Porcentaje de NA por columna:
Fecha          0.0
CO_mg_m3      10.5
NO_ug_m3       9.6
NO2_ug_m3      9.5
O3_ug_m3       9.0
PM10_ug_m3    10.1
SO2_ug_m3     10.3
Provincia      0.0
Estacion       0.0
dtype: float64
```

Figura 11: Estadísticas de valores ausentes.

3.3.2. Tratamiento de datos ausentes

Existen varias maneras de tratar con valores ausentes:

- Rellenar los valores con la media, mediana o el valor más frecuente de la variable.
- Completar los valores que faltan por el valor que esté directamente antes o después en la fila o columna.
- Completar todos los datos faltantes con 0, si se trata de valores numéricos. Esta opción es poco aconsejable ya que puedes modificar de manera significativa los resultados.
- Eliminar las filas que presenten valores ausentes, siempre y cuando el conjunto de datos sea lo suficientemente grande y no se pierde información relevante al eliminar esas filas.
- Una forma abrupta de tratamiento que depende del contexto de análisis, es eliminar las variables que presentan un porcentaje mayor del 50 % de datos ausentes.

```
# Crear una copia para no modificar el original
calidad_aire_clean = calidad_aire.copy()

# 1. Eliminar variables con >50% de NA
umbral = 0.5
columnas_a_eliminar = []
```

```python
for col in calidad_aire_clean.columns:
    if col != 'Fecha':  # Excluimos la fecha de este analisis
        na_porcentaje = calidad_aire_clean[col].isna().mean()
        if na_porcentaje > umbral:
            columnas_a_eliminar.append(col)

print(f'Columnas con >{umbral*100}% NA: {columnas_a_eliminar}')
if columnas_a_eliminar:
    calidad_aire_clean = calidad_aire_clean.drop(columns=
        columnas_a_eliminar)

# 2. Sustituir NA por la media (solo para columnas numericas)
columnas_numericas = calidad_aire_clean.select_dtypes(include=[np.number
    ]).columns

for col in columnas_numericas:
    media = calidad_aire_clean[col].mean(skipna=True)
    calidad_aire_clean[col] = calidad_aire_clean[col].fillna(round(media
        , 2))

print('\nValores NA despues del tratamiento:',
      calidad_aire_clean.isna().any().any())
```

Listing 6: Tratamiento de valores ausentes

3.4. Detección y tratamiento de valores atípicos (outliers)

Figura 12: Etapa N° 4 - Identificación de valores atípicos.

Un valor atípico (outlier), es una observación significativamente distinta del resto de datos que presenta una variable, de tal magnitud que se puede considerar un valor anómalo. Estos valores pueden afectar a tareas futuras pudiendo llegar a modificar los resultados. Es necesario su detección y tratamiento a fin de disminuir su incidencia en esos análisis o bien, **en casos muy extremos**, eliminarlos del dataset.

3.4.1. Variables continuas

```
# Seleccionamos una variable para el ejemplo
variable = 'O3_ug_m3'
datos = calidad_aire_clean[variable].copy()

# Visualizacion con boxplot
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histograma
axes[0].hist(datos, bins=50, alpha=0.7, color='skyblue', edgecolor='
    black')
axes[0].set_xlabel(f'{variable} (ug/m3)')
axes[0].set_ylabel('Frecuencia')
axes[0].set_title(f'Distribucion de {variable}')

# Boxplot
axes[1].boxplot(datos, vert=False)
axes[1].set_xlabel(f'{variable} (ug/m3)')
axes[1].set_title(f'Boxplot de {variable}')

```

```python
plt.tight_layout()
plt.show()

# Calculo de estadisticos para detectar outliers
Q1 = datos.quantile(0.25)
Q3 = datos.quantile(0.75)
IQR = Q3 - Q1
limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

outliers = datos[(datos < limite_inferior) | (datos > limite_superior)]

print(f'  Estadisticas para {variable}:')
print(f'  Q1: {Q1:.2f}')
print(f'  Mediana: {datos.median():.2f}')
print(f'  Q3: {Q3:.2f}'')
print(f'  IQR: {IQR:.2f}'')
print(f'  Limite inferior: {limite_inferior:.2f}')
print(f'  Limite superior: {limite_superior:.2f}')
print(f'  Numero de observaciones: {len(datos)}')
print(f'  Numero de outliers: {len(outliers)}')
print(f'  Porcentaje de outliers: {(len(outliers)/len(datos))*100:.2f}\%
    ')
```

Listing 7: Detección de outliers con boxplots

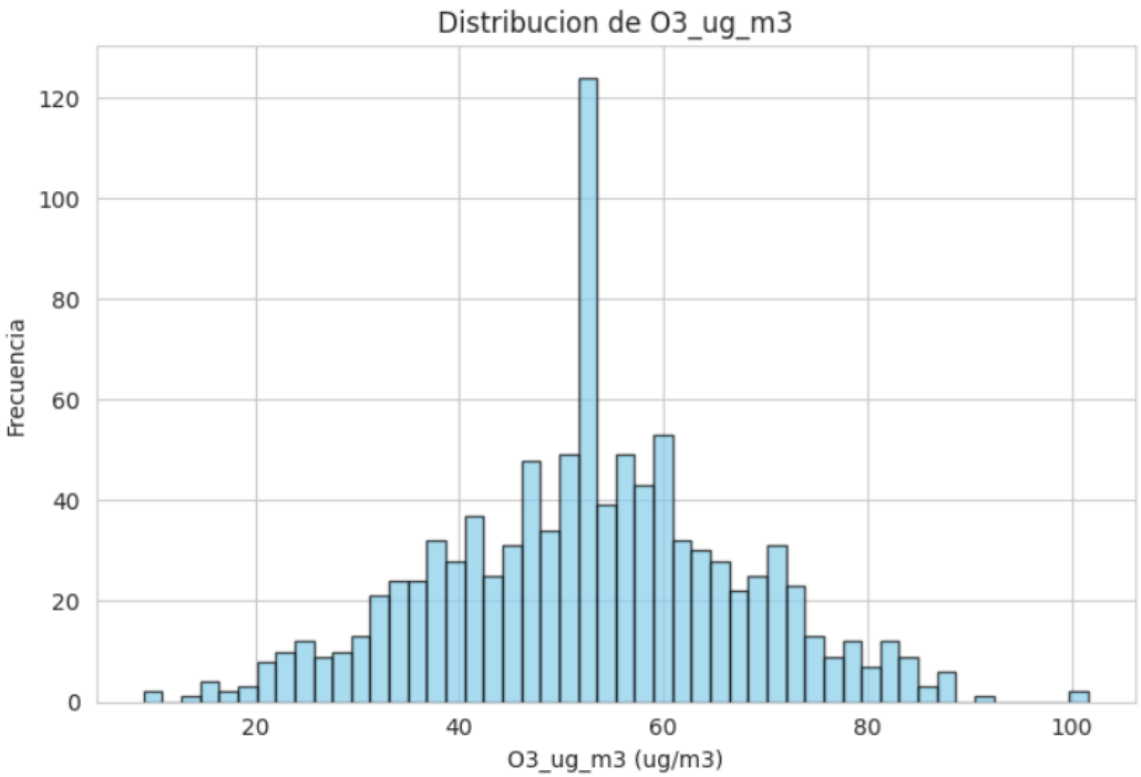


Figura 13: Representación del histograma de la variable O3.

Los gráficos de cajas y bigotes (boxplots), aportan una representación visual que describe la dispersión y simetría que presentan los datos observando los cuartiles (división de la distribución en cuatro partes delimitadas por los valores 0.25, 0.50 y 0.75). Estos gráficos están compuestos por tres componentes:

- **Rango intercuartil (IQR):**
  Representa el 50 % de los datos, comprende desde el percentil 25 de la distribución (**Q1**), hasta el percentil 75 (**Q3**). Dentro de la caja se presenta una línea que señala el percentil 50 de la distribución (**Q2**), la mediana. La caja aporta una idea sobre la dispersión de la distribución en función de la separación existente entre **Q1** y **Q3**, así como también si la distribución es simétrica en torno a la mediana o si está sesgada hacia alguno de los lados.
- **Bigotes:**
  Se extienden desde ambos lados de los extremos de la caja y representan los rangos del 25 % de valores de la parte inferior ($Q1 - 1{,}5 \times IQR$) y el 25 % de valores de la parte superior ($Q3 + 1{,}5 \times IQR$), excluyendo los valores atípicos.
- **Valores atípicos:**
  Esta representación identifica como valores atípicos aquellas observaciones que presentan

valores inferiores o superiores a los límites del gráfico (límite inferior: $Q1\breve{}1{,}5 \times IQR$ y límite superior: $Q3 + 1{,}5 \times IQR$).

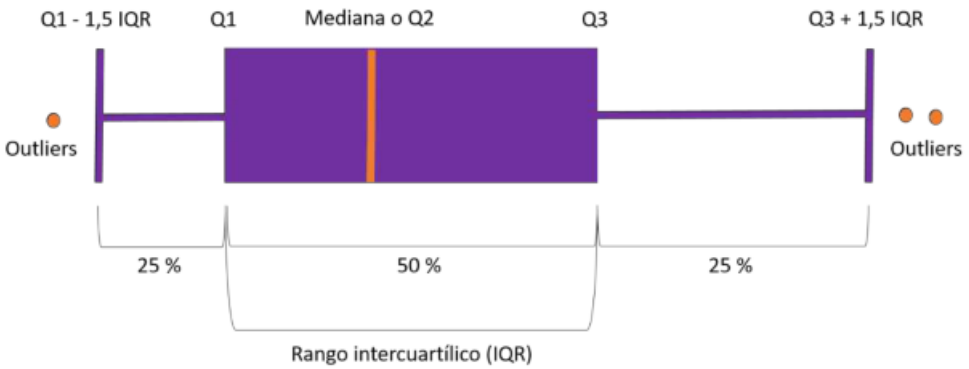


Figura 14: Representación de un gráfico boxplot para identificar la presencia de valores atípicos.

```
# Eliminar outliers
calidad_aire_no_out = calidad_aire_clean[
    (calidad_aire_clean[variable] >= limite_inferior) &
    (calidad_aire_clean[variable] <= limite_superior)
].copy()

print(f''Dimensiones originales: {calidad_aire_clean.shape}'')
print(f''Dimensiones sin outliers: {calidad_aire_no_out.shape}'')
print(f''Filas eliminadas: {len(calidad_aire_clean) - len(
    calidad_aire_no_out)}'')

# Comparacion de boxplots antes y despues
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].boxplot(calidad_aire_clean[variable], vert=False)
axes[0].set_xlabel(f'{variable} (ug/m3)')
axes[0].set_title(f'Antes de eliminar outliers')

axes[1].boxplot(calidad_aire_no_out[variable], vert=False)
axes[1].set_xlabel(f'{variable} (ug/m3)')
axes[1].set_title(f'Despues de eliminar outliers')

```

```
plt.tight_layout()
plt.show()
```

Listing 8: Eliminación de outliers

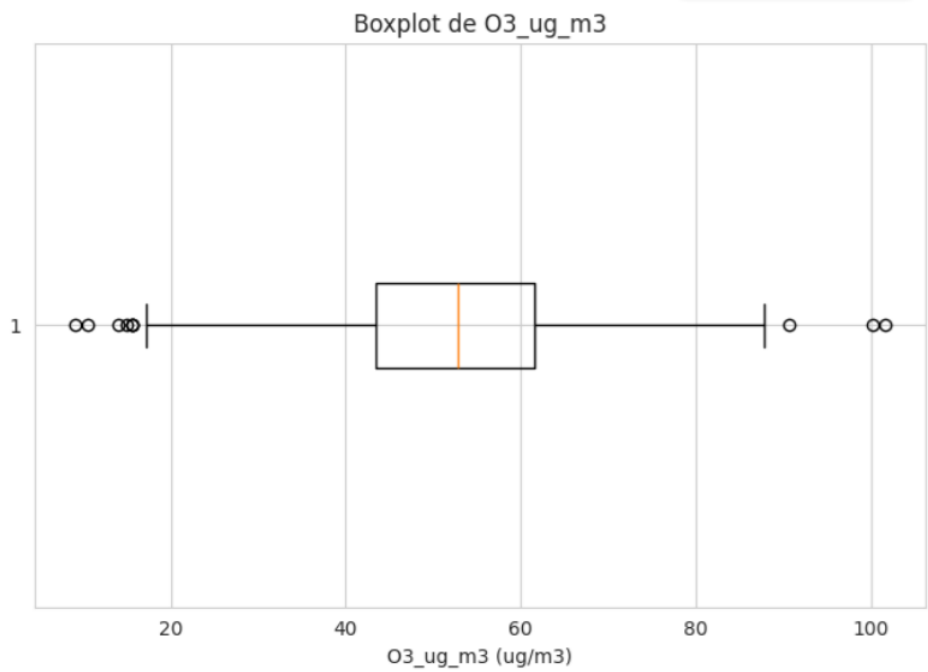


Figura 15: Representación de un gráfico boxplot en el que se observa la presencia de valores atípicos y su distribución. Variable O3.

```
Estadisticas para O3_ug_m3:
Q1: 43.50
Mediana: 52.78
Q3: 61.61
IQR: 18.10
Limite inferior: 16.35
Limite superior: 88.76
Numero de observaciones: 1000
Numero de outliers: 10
Porcentaje de outliers: 1.00%
```

Figura 16: Estadísticas de la variable O3.

3.4.2. Variables categóricas

```
# Para variables categoricas, usamos graficos de barras
# Para variables categoricas, usamos graficos de barras
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Con matplotlib
conteo_provincias = calidad_aire_clean['Provincia'].value_counts()
axes[0].bar(conteo_provincias.index, conteo_provincias.values,
            color='skyblue', edgecolor='black')
axes[0].set_xlabel('Provincia')
axes[0].set_ylabel('Numero de observaciones')
axes[0].set_title('Distribucion por provincia (matplotlib)')
axes[0].tick_params(axis='x', rotation=45)

# Con seaborn (mas estetico)
sns.countplot(data=calidad_aire_clean, x='Provincia', hue='Provincia', ax=axes[1],
              palette='viridis', legend=False)
axes[1].set_xlabel('Provincia')
axes[1].set_ylabel('Numero de observaciones')
axes[1].set_title('Distribucion por provincia (seaborn)')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

# Deteccion de categorias problematicas
print('Frecuencia por provincia:')
print(calidad_aire_clean['Provincia'].value_counts())

# Identificar posibles outliers categoricos (como 'Madrid' en el ejemplo)
categorias_raras = ['Madrid']  # En un caso real, detectariamos automaticamente
for cat in categorias_raras:
    if cat in calidad_aire_clean['Provincia'].values:
        print(f'\nCategoria {cat} detectada con {sum(calidad_aire_clean['Provincia'] == cat)} observaciones')

```

```python
        # Eliminar la categoria problematica
        calidad_aire_sm = calidad_aire_clean[calidad_aire_clean['Provincia'] != cat].copy()
        print(f'Datos despues de eliminar {cat}: {len(calidad_aire_sm)} observaciones')

        # Verificar que se elimino la categoria
        print('Categorias restantes:', calidad_aire_sm['Provincia'].unique())
```

Listing 9: Análisis de variables categóricas

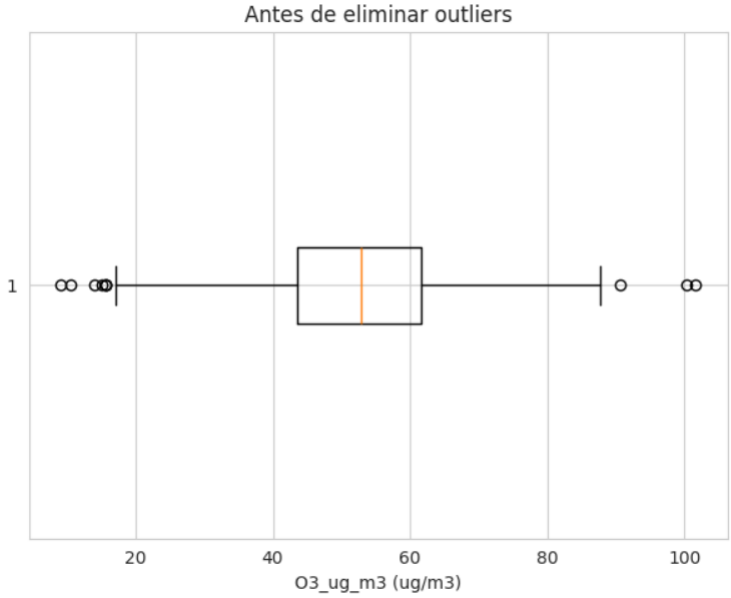


Figura 17: Antes de eliminar outliers.

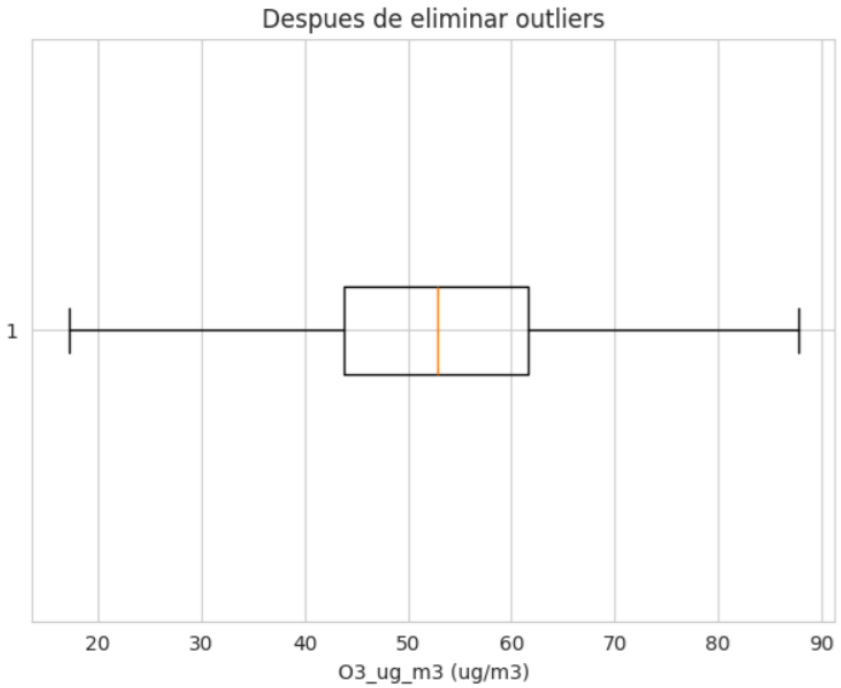


Figura 18: Después de eliminar outliers.

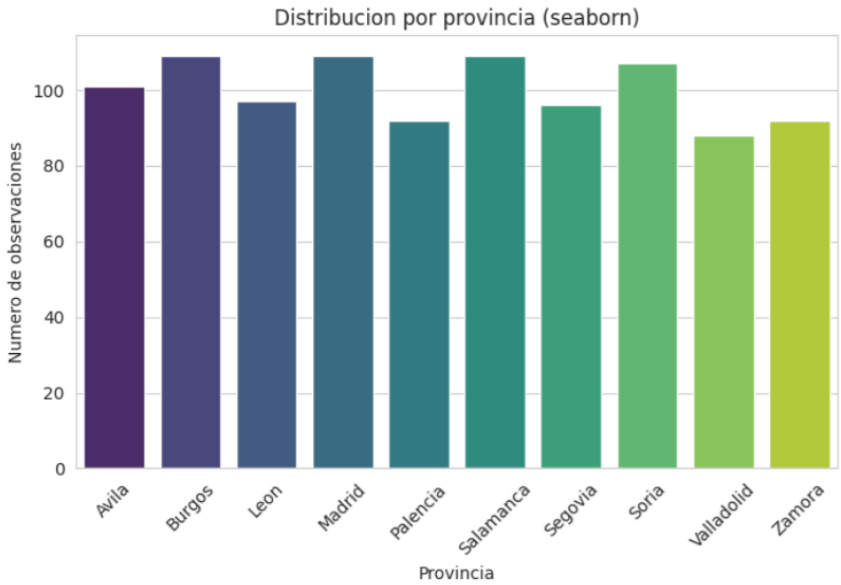


Figura 19: Distribución por provincia (librería `seaborn`).

3.5. Análisis de correlación entre variables

Figura 20: Etapa N° 5 - Correlación de variables.

El análisis de correlación permite identificar relaciones lineales entre variables numéricas. Esto no quiere decir, que una correlación entre variables indique una relación causa-efecto. La existencia de una relación fuerte en un determinado sentido entre dos variables podría inferir redundancia de información, pudiendo llegar a la eliminación de una de ellas con el fin de disminuir la complejidad en el procesamiento y análisis futuro de los datos. Esta práctica es habitual en EDA y está vinculada con la técnica de análisis de componentes principales.

La correlación se mide a través del coeficiente de correlación ”r” que toma valores entre **-1** y **1**, a saber:

- **Correlación positiva perfecta**: se establece con el valor **1** e indica que los valores de las variables varían de una forma similar.
- **Correlación negativa perfecta**: se establece con el valor **–1**, indicando que varían de forma inversa.
- **Correlación nula**: Las variables son independientes cuando el coeficiente es **0**

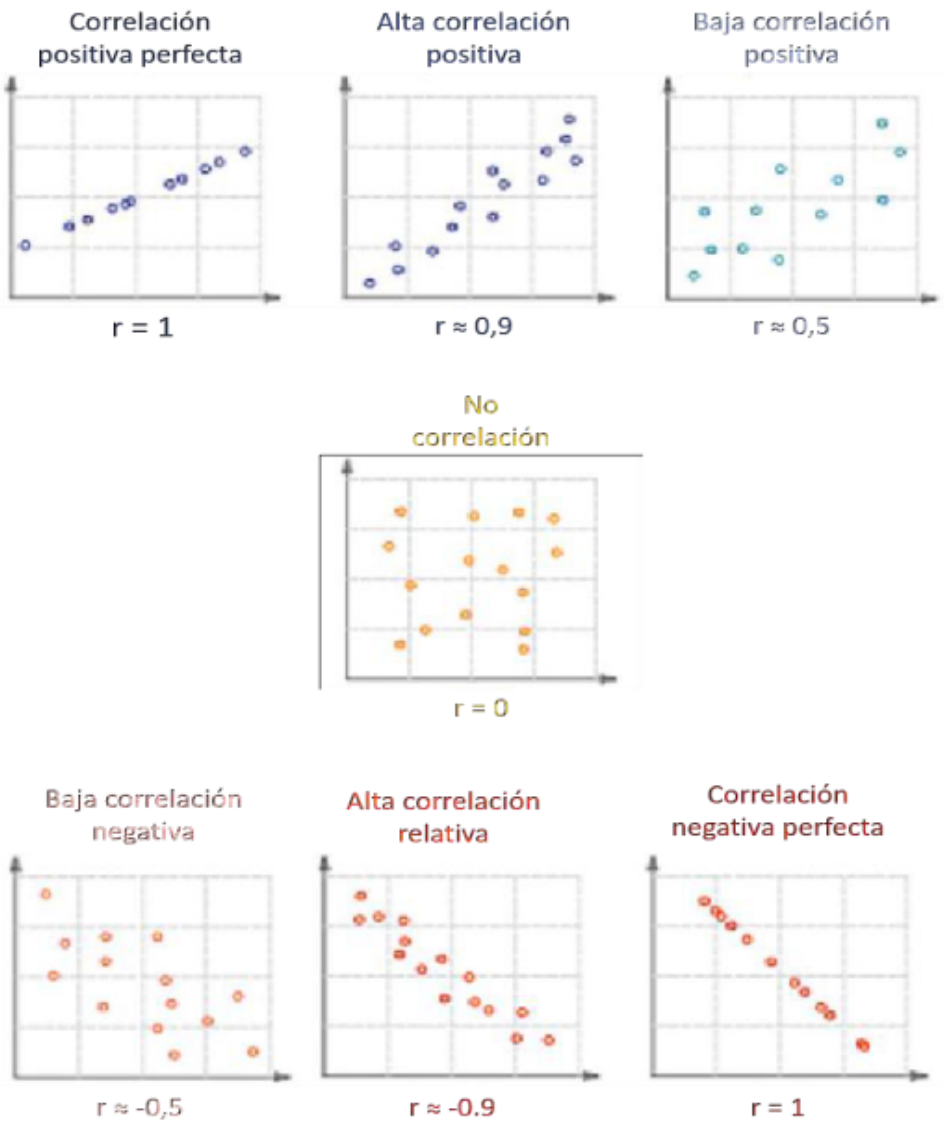


Figura 21: Interpretación entre la correlación lineal entre dos variables.

```
# Seleccionar variables numericas
variables_numericas = calidad_aire_clean.select_dtypes(include=[np.
    number]).columns
print('Variables numericas:', list(variables_numericas))

# Calcular matriz de correlacion
matriz_correlacion = calidad_aire_clean[variables_numericas].corr()

print('\nMatriz de correlacion:')
```

```python
print(matriz_correlacion.round(3))

# Visualizacion con heatmap (similar a corrplot en R)
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Heatmap con seaborn
sns.heatmap(matriz_correlacion, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=1, ax=axes[0], fmt='.2f',
            cbar_kws={'shrink': 0.8})
axes[0].set_title('Matriz de correlacion (heatmap)')

# Visualizacion alternativa con circulos (similar al ejemplo de corrplot)
from matplotlib.patches import Circle
import matplotlib.patches as mpatches

ax = axes[1]
n_vars = len(variables_numericas)
ax.set_xlim(-0.5, n_vars - 0.5)
ax.set_ylim(-0.5, n_vars - 0.5)
ax.set_aspect('equal')

for i in range(n_vars):
    for j in range(n_vars):
        corr = matriz_correlacion.iloc[i, j]
        size = abs(corr) * 0.8  # Tamanio proporcional a la correlacion
        color = 'red' if corr > 0 else 'blue'
        alpha = abs(corr)
        circle = Circle((j, n_vars-1-i), size/2, color=color, alpha=alpha)
        ax.add_patch(circle)

ax.set_xticks(range(n_vars))
ax.set_yticks(range(n_vars))
ax.set_xticklabels(variables_numericas, rotation=45, ha='right')
ax.set_yticklabels(reversed(variables_numericas))
ax.set_title('Matriz de correlacion (circulos proporcionales)')

# Leyenda
```

```python
red_patch = mpatches.Patch(color='red', alpha=0.6, label='Correlacion positiva')
blue_patch = mpatches.Patch(color='blue', alpha=0.6, label='Correlacion negativa')
ax.legend(handles=[red_patch, blue_patch], loc='lower right')

plt.tight_layout()
plt.show()

# Analisis de correlaciones altas
corr_alta = []
for i in range(len(variables_numericas)):
    for j in range(i+1, len(variables_numericas)):
        if abs(matriz_correlacion.iloc[i, j]) > 0.7:
            corr_alta.append((variables_numericas[i],
                              variables_numericas[j],
                              matriz_correlacion.iloc[i, j]))

print('\nPares con correlacion alta (>0.7):')
for par in corr_alta:
    print(f'  {par[0]} - {par[1]}: r = {par[2]:.3f}')
```

Listing 10: Matriz de correlación y visualización

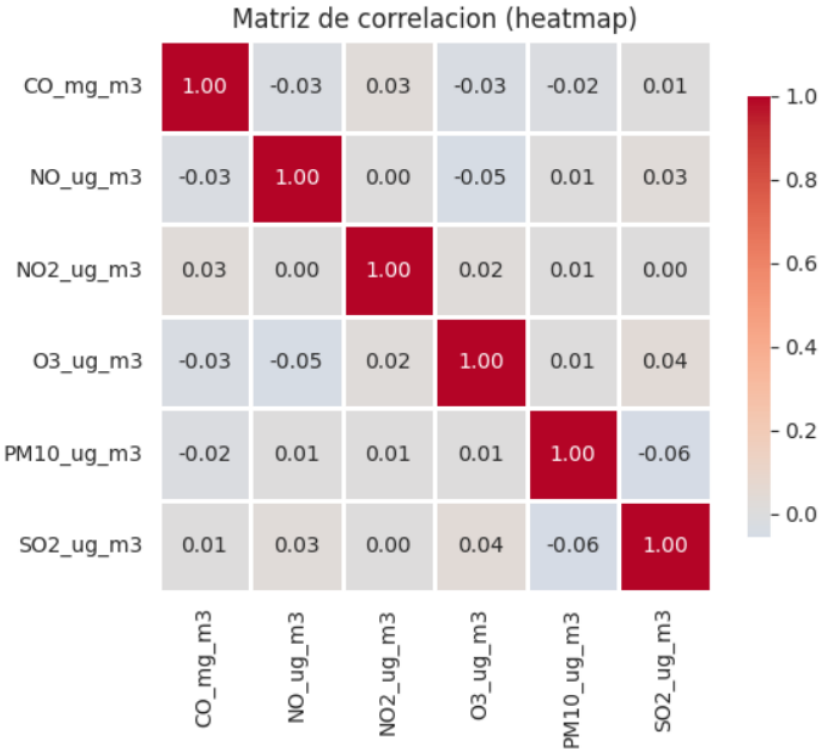


Figura 22: Matriz de correlación.

```python
# Pairplot para visualizar relaciones multiples
g = sns.pairplot(calidad_aire_clean[variables_numericas],
                 diag_kind='hist',
                 plot_kws={'alpha': 0.6, 's': 20, 'edgecolor': 'none'},
                 diag_kws={'alpha': 0.6, 'bins': 30})
g.fig.suptitle('Relaciones entre variables numericas', y=1.02)
plt.tight_layout()
plt.show()

# Scatter plot especifico para variables con alta correlacion
if corr_alta:
    var1, var2, r_val = corr_alta[0]
    fig, ax = plt.subplots(figsize=(10, 6))

    scatter = ax.scatter(calidad_aire_clean[var1],
                         calidad_aire_clean[var2],
                         alpha=0.5, c=calidad_aire_clean.index,
                         cmap='viridis', s=30)
```

```python

    # Linea de regresion
    z = np.polyfit(calidad_aire_clean[var1], calidad_aire_clean[var2],
        1)
    p = np.poly1d(z)
    ax.plot(calidad_aire_clean[var1].sort_values(),
            p(calidad_aire_clean[var1].sort_values()),
            'r--', alpha=0.8, label=f'Tendencia lineal (r={r_val:.3f})')

    ax.set_xlabel(var1)
    ax.set_ylabel(var2)
    ax.set_title(f'Relacion entre {var1} y {var2}')
    ax.legend()
    ax.grid(True, alpha=0.3)

    plt.colorbar(scatter, ax=ax, label='Indice')
    plt.tight_layout()
    plt.show()
```

Listing 11: Visualizacion de relaciones bivariadas

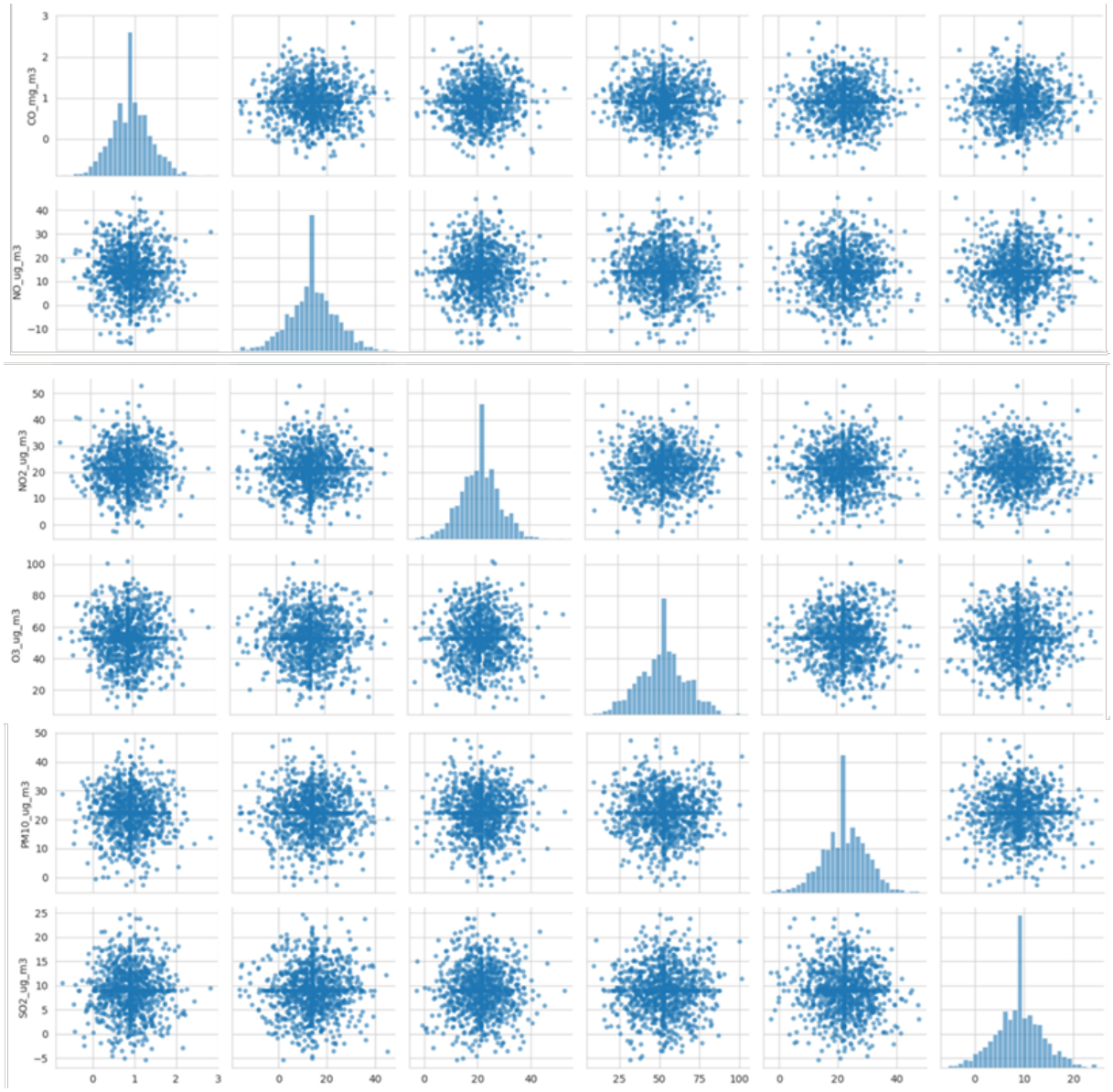


Figura 23: Relaciones entre variables numéricas.

4. Conclusiones

El Análisis Exploratorio de Datos (EDA) constituye una fase crítica en cualquier proyecto de ciencia de datos. A través de este recorrido práctico con Python, hemos demostrado cómo implementar las técnicas fundamentales de EDA:

- **Análisis descriptivo:** Comprensión inicial de la estructura y distribución de los datos mediante funciones como `info()`, `describe()` y visualizaciones básicas.
- **Ajuste de tipos:** Verificación y conversión de variables a los tipos adecuados (datetime, categórico, numérico).
- **Manejo de datos ausentes:** Detección sistemática de valores perdidos y aplicación de estrategias como eliminación de variables con alto porcentaje de **NA** o imputación por la media.
- **Tratamiento de outliers:** Identificación mediante boxplots y criterios estadísticos (IQR), con decisiones documentadas sobre su eliminación o tratamiento.
- **Análisis de correlaciones:** Exploración de relaciones lineales entre variables mediante matrices de correlación y visualizaciones avanzadas.

Las herramientas de Python (`pandas`, `numpy`, `matplotlib`, `seaborn`) realizan estas tareas, con la ventaja de integrarse en un ecosistema más amplio de ciencia de datos y machine learning.

5. Recursos adicionales

Para profundizar en el análisis exploratorio de datos con Python, se recomiendan los siguientes recursos:

- **Libros gratuitos:**
  - ”Python for Data Analysis” - Wes McKinney
  - ”Python Data Science Handbook” - Jake VanderPlas

- **Cheat sheets útiles:**
  - Pandas Cheat Sheet
  - Matplotlib Cheat Sheet
  - Seaborn Cheat Sheet
  - Scikit-learn Cheat Sheet

Esta guía ha sido adaptada del documento original Guía práctica de introducción al Análisis Exploratorio de Datos, perteneciente a la Iniciativa Aporta del Gobierno de España, manteniendo la estructura pedagógica pero utilizando el ecosistema Python, ampliamente adoptado en la industria y la academia para ciencia de datos.